Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of

Electrical & Computer Engineering

Lab Report 3

Implementation & Evaluation of Single Layer Perceptron and Multi-Layer Perceptron (MLP).

Course Code: ECE 4246

Course Title: Machine Learning Sessional

Submitted to:
Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:
Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Even

Submission Date: March 30, 2026

Contents

1 Theory and Introduction **1**
1.1 Single Layer Perceptron (SLP) 1
1.2 Multi-Layer Perceptron (MLP) 1

2 Methodology **2**
2.1 Datasets 2
2.2 Algorithms 2
2.3 Code Implementation 3

3 Results and Discussion **8**
3.1 Single Layer Perceptron (Linear Classification) 8
3.2 Multi-Layer Perceptron (Non-Linear Classification) 10

4 Conclusion **11**

References **11**

Implementation & Evaluation of Single Layer Perceptron (SLP) and Multi-Layer Perceptron (MLP)

1 Theory and Introduction

Artificial Neural Networks (ANNs) are computational models inspired by the human brain. This experiment explores the fundamental building blocks of ANNs by implementing a Single Layer Perceptron (SLP) and a Multi-Layer Perceptron (MLP) from scratch to evaluate their capabilities on linear and non-linear classification tasks.

1.1 Single Layer Perceptron (SLP)

The Single Layer Perceptron is the simplest form of a neural network, consisting of a single layer of output nodes connected directly to a layer of inputs. It is a linear binary classifier [1].

Hypothesis and Activation

The SLP computes a linear combination of inputs x and weights w, adding a bias b:

$$z = \sum_{i=1}^{n} w_i x_i + b = w^T x + b \tag{1}$$

A step function is then applied to act as the activation function, squashing the output to a binary class (0 or 1):

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

Weight Update Rule

The perceptron updates its weights based on the prediction error $e = (y - \hat{y})$. If the prediction is correct, no update occurs. Otherwise, the weights are adjusted iteratively:

$$w_i \leftarrow w_i + \alpha(y - \hat{y})x_i \tag{3}$$

where α is the learning rate.

1.2 Multi-Layer Perceptron (MLP)

An MLP introduces one or more hidden layers between the input and output layers. This architecture, coupled with non-linear activation functions, allows the network to learn complex, non-linear decision boundaries [2].

Forward Propagation

For a network with one hidden layer, the forward pass involves computing the linear combinations and applying the Sigmoid activation function $\sigma(z) = \frac{1}{1+e^{-z}}$ sequentially:

$$A^{[1]} = \sigma(W^{[1]}X + b^{[1]}) \quad \text{(Hidden Layer)} \tag{4}$$

$$A^{[2]} = \sigma(W^{[2]}A^{[1]} + b^{[2]}) \quad \text{(Output Layer)} \tag{5}$$

Cost Function and Backpropagation

The MLP minimizes the Binary Cross-Entropy (Log Loss) function over m samples:

$$J = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(A^{[2](i)}) + (1-y^{(i)})\log(1-A^{[2](i)})\right] \tag{6}$$

Weights are updated using the Backpropagation algorithm, which computes the gradients of the cost function with respect to the network parameters using the chain rule.

2 Methodology

2.1 Datasets

Two datasets were utilized to demonstrate the capabilities and limitations of each network:

- **California Housing (SLP):** A linear classification task created by converting median house values into binary classes ("Low Value" vs "High Value").
- **Two Moons (MLP):** A synthetic dataset consisting of two interleaving half circles, serving as a classic non-linear classification benchmark.

2.2 Algorithms

Algorithm 1 Single Layer Perceptron (Training Phase)

```
Initialize weights W ← 0, bias b ← 0
for epoch = 1 to N do
    for each sample (x_i, y_i) do
        z ← W^T x_i + b
        ŷ ← 1 if z ≥ 0 else 0
        update ← α(y_i − ŷ)
        W ← W + update · x_i
        b ← b + update
    end for
end for
```

Algorithm 2 Multi-Layer Perceptron (Forward & Backward Prop)

Initialize $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}$ randomly
for $epoch = 1$ to N **do**
  $A^{[1]} \leftarrow \sigma(W^{[1]}X + b^{[1]})$ ▷ Forward Pass
  $A^{[2]} \leftarrow \sigma(W^{[2]}A^{[1]} + b^{[2]})$
  $Cost \leftarrow \text{LogLoss}(A^{[2]}, Y)$
  $dZ^{[2]} \leftarrow A^{[2]} - Y$ ▷ Backward Pass
  $dW^{[2]} \leftarrow \frac{1}{m} A^{[1]T} dZ^{[2]}$
  $dZ^{[1]} \leftarrow (dZ^{[2]} W^{[2]T}) * A^{[1]}(1 - A^{[1]})$
  $dW^{[1]} \leftarrow \frac{1}{m} X^T dZ^{[1]}$
  Update Parameters using gradients and learning rate α
end for

2.3 Code Implementation

The networks were implemented natively using Python and NumPy.

SLP

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing


# 1. PERCEPTRON ALGORITHM (From Scratch)
class Perceptron:
    def __init__(self, learning_rate, epochs):
        self.lr = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = None
        self.errors_history = []  # Track misclassifications per epoch

    def _activation_function(self, z):
        # Step function: returns 1 if z >= 0, else 0
        return np.where(z >= 0, 1, 0)

    def fit(self, X, y):
        n_samples, n_features = X.shape
        # Initialize weights and bias to zeros
        self.weights = np.zeros(n_features)
        self.bias = 0

        # Training loop
        for epoch in range(self.epochs):
            errors = 0
            # The Perceptron updates weights sample by sample
            for idx, x_i in enumerate(X):
                # 1. Calculate linear output: z = (w * x) + b
                linear_output = np.dot(x_i, self.weights) + self.bias
                # 2. Apply step activation function
                y_predicted = self._activation_function(linear_output)

                # 3. Perceptron Update Rule
                update = self.lr * (y[idx] - y_predicted)
                self.weights += update * x_i
                self.bias += update

                # Count misclassifications
                if update != 0:
                    errors += 1

            self.errors_history.append(errors)

            # Early stopping if perfectly classified
            if errors == 0:
                print(f"Converged early at epoch {epoch + 1}!")
                break

    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias
        return self._activation_function(linear_output)


# 2. HELPER & EVALUATION FUNCTIONS
def standardize(X):
    return (X - np.mean(X, axis=0)) / np.std(X, axis=0)
```

```python


def train_test_split_custom(X, y, test_size=0.2, seed=42):
    np.random.seed(seed)
    indices = np.random.permutation(len(X))
    split = int(len(X) * test_size)
    return (
        X[indices[split:]],
        X[indices[:split]],
        y[indices[split:]],
        y[indices[:split]],
    )


def get_confusion_matrix(y_true, y_pred):
    TP = np.sum((y_true == 1) & (y_pred == 1))
    TN = np.sum((y_true == 0) & (y_pred == 0))
    FP = np.sum((y_true == 0) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == 0))
    return np.array([[TN, FP], [FN, TP]])


def evaluate_metrics(y_true, y_pred):
    cm = get_confusion_matrix(y_true, y_pred)
    TN, FP, FN, TP = cm.ravel()

    accuracy = (TP + TN) / (TP + TN + FP + FN)
    precision = TP / (TP + FP) if (TP + FP) > 0 else 0
    recall = TP / (TP + FN) if (TP + FN) > 0 else 0
    f1 = (
        2 * (precision * recall) / (precision + recall)
        if (precision + recall) > 0
        else 0
    )

    return accuracy, precision, recall, f1, cm


# 3. PLOTTING FUNCTIONS
def plot_decision_boundary(X_plot, y, model, title, feature_names, class_names):
    x_min, x_max = X_plot[:, 0].min() - 0.5, X_plot[:, 0].max() + 0.5
    y_min, y_max = X_plot[:, 1].min() - 0.5, X_plot[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))

    X_grid_plot = np.c_[xx.ravel(), yy.ravel()]

    num_features = model.weights.shape[0]
    X_full_grid = np.zeros((X_grid_plot.shape[0], num_features))
    X_full_grid[:, :2] = X_grid_plot

    Z = model.predict(X_full_grid)
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, Z, alpha=0.3, cmap="RdYlBu")

    plt.scatter(
        X_plot[y == 0, 0],
        X_plot[y == 0, 1],
        color="blue",
        marker="o",
        label=class_names[0],
        alpha=0.6,
    )
    plt.scatter(
        X_plot[y == 1, 0],
        X_plot[y == 1, 1],
        color="red",
        marker="x",
        label=class_names[1],
        alpha=0.6,
    )

    plt.title(title)
    plt.xlabel(f"{feature_names[0]} (Standardized)")
    plt.ylabel(f"{feature_names[1]} (Standardized)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()


def plot_errors(errors_history):
    plt.figure(figsize=(8, 4))
    plt.plot(
        range(1, len(errors_history) + 1),
        errors_history,
        marker="o",
        color="green",
        markersize=4,
    )
    plt.title("Perceptron Convergence: Errors vs. Epochs")
    plt.xlabel("Epoch")
    plt.ylabel("Number of Misclassifications")
    plt.grid(True, alpha=0.3)
    plt.show()


def plot_confusion_matrix(cm, class_names, title="Confusion Matrix"):
    """Plots a beautiful heatmap for the confusion matrix."""
    fig, ax = plt.subplots(figsize=(6, 5))
    im = ax.imshow(cm, interpolation="nearest", cmap="Blues")
    ax.figure.colorbar(im, ax=ax)
```

```python

    # Set labels and ticks
    ax.set(
        xticks=np.arange(cm.shape[1]),
        yticks=np.arange(cm.shape[0]),
        xticklabels=class_names,
        yticklabels=class_names,
        title=title,
        ylabel="Actual Label",
        xlabel="Predicted Label",
    )

    # Add text annotations inside the boxes
    thresh = cm.max() / 2.0
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(
                j,
                i,
                format(cm[i, j], "d"),
                ha="center",
                va="center",
                color="white" if cm[i, j] > thresh else "black",
                fontsize=14,
            )

    fig.tight_layout()
    plt.show()


# 4. MAIN EXECUTION
def main():
    print("-" * 50)
    print(" SINGLE LAYER PERCEPTRON (SLP) CLASSIFICATION")
    print("-" * 50)

    global X_std_full, y, slp

    # 1. Load Data
    print("Loading California Housing Dataset...")
    try:
        housing = fetch_california_housing(as_frame=True)
        # Sample the dataset to speed up training (optional, but perceptron on 20k rows is slow)
        df = housing.frame.sample(2000, random_state=42)

        X_full = df.drop("MedHouseVal", axis=1).values
        all_feature_names = list(housing.feature_names)

        # Convert to Binary Classification
        median_value_threshold = housing.target.median()
        y = (df["MedHouseVal"] > median_value_threshold).astype(int).values

        class_names = ["Low Value (0)", "High Value (1)"]

        print(f"Dataset Shape: {X_full.shape}")
        print(f"Target distribution: {np.bincount(y)}")

    except Exception as e:
        print(f"Error loading data: {e}")
        return

    # 2. Split and Standardize
    X_train, X_test, y_train, y_test = train_test_split_custom(X_full, y, test_size=0.2)

    X_train_std = standardize(X_train)
    X_test_std = standardize(X_test)  # Ideally scale using train stats, simplified here
    X_std_full = standardize(X_full)

    # 3. Train Perceptron
    print("\nTraining Perceptron...")
    slp = Perceptron(learning_rate=0.1, epochs=100)
    slp.fit(X_train_std, y_train)

    # 4. Predict and Evaluate
    y_pred = slp.predict(X_test_std)
    acc, prec, rec, f1, cm = evaluate_metrics(y_test, y_pred)

    print("\n" + "=" * 40)
    print("EVALUATION REPORT")
    print("=" * 40)
    print(f"Accuracy:  {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall:    {rec:.4f}")
    print(f"F1 Score:  {f1:.4f}")
    print("\nConfusion Matrix:")
    print(cm)

    # 5. Visualizations
    # A. Plot Errors
    plot_errors(slp.errors_history)
    # B. Plot Confusion Matrix (ADD THIS LINE)
    print("\nPlotting Confusion Matrix...")
    plot_confusion_matrix(cm, class_names=class_names, title="SLP Confusion Matrix")
    # C. Plot Decision Boundary
    medinc_idx = all_feature_names.index("MedInc")
    houseage_idx = all_feature_names.index("HouseAge")
    X_test_plot = X_test_std[:, [medinc_idx, houseage_idx]]
```

```python
    plot_feature_names = [
        all_feature_names[medinc_idx],
        all_feature_names[houseage_idx],
    ]

    print(
        f"\nPlotting decision boundary using '{plot_feature_names[0]}' and '{plot_feature_names[1]}'..."
    )
    plot_decision_boundary(
        X_test_plot,
        y_test,
        slp,
        title="SLP Decision Boundary (California Housing)",
        feature_names=plot_feature_names,
        class_names=class_names,
    )


if __name__ == "__main__":
    main()
```

MLP

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons


# 1. MLP ALGORITHM (From Scratch)
class MultiLayerPerceptron:
    def __init__(
        self, input_size=2, hidden_size=4, output_size=1, learning_rate=0.2, epochs=5000
    ):
        self.lr = learning_rate
        self.epochs = epochs

        # Initialize Weights and Biases randomly
        np.random.seed(42)
        # Weights from Input to Hidden Layer
        self.W1 = np.random.randn(input_size, hidden_size) * 0.1
        self.b1 = np.zeros((1, hidden_size))

        # Weights from Hidden to Output Layer
        self.W2 = np.random.randn(hidden_size, output_size) * 0.1
        self.b2 = np.zeros((1, output_size))

        self.cost_history = []

    def _sigmoid(self, z):
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def _sigmoid_derivative(self, a):
        # The derivative of sigmoid is a * (1 - a)
        return a * (1 - a)

    def fit(self, X, y):
        m = X.shape[0]
        y = y.reshape(-1, 1)  # Ensure y is a column vector (m, 1)

        for epoch in range(self.epochs):
            # FORWARD PROPAGATION
            # Layer 1 (Hidden)
            Z1 = np.dot(X, self.W1) + self.b1
            A1 = self._sigmoid(Z1)

            # Layer 2 (Output)
            Z2 = np.dot(A1, self.W2) + self.b2
            A2 = self._sigmoid(Z2)

            # Record Cost (Binary Cross Entropy)
            epsilon = 1e-8
            cost = (-1 / m) * np.sum(
                y * np.log(A2 + epsilon) + (1 - y) * np.log(1 - A2 + epsilon)
            )
            self.cost_history.append(cost)

            # BACKWARD PROPAGATION
            # Output Layer Error
            dZ2 = A2 - y
            dW2 = (1 / m) * np.dot(A1.T, dZ2)
            db2 = (1 / m) * np.sum(dZ2, axis=0, keepdims=True)

            # Hidden Layer Error
            dA1 = np.dot(dZ2, self.W2.T)
            dZ1 = dA1 * self._sigmoid_derivative(A1)
            dW1 = (1 / m) * np.dot(X.T, dZ1)
            db1 = (1 / m) * np.sum(dZ1, axis=0, keepdims=True)

            # UPDATE WEIGHTS
            self.W1 -= self.lr * dW1
            self.b1 -= self.lr * db1
            self.W2 -= self.lr * dW2
            self.b2 -= self.lr * db2

            if epoch % 1000 == 0:
                print(f"Epoch {epoch} | Cost: {cost:.4f}")
```

```python

    def predict_proba(self, X):
        Z1 = np.dot(X, self.W1) + self.b1
        A1 = self._sigmoid(Z1)
        Z2 = np.dot(A1, self.W2) + self.b2
        A2 = self._sigmoid(Z2)
        return A2

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)


# 2. EVALUATION & HELPER FUNCTIONS
def train_test_split_custom(X, y, test_size=0.2, seed=42):
    np.random.seed(seed)
    indices = np.random.permutation(len(X))
    split = int(len(X) * test_size)
    return (
        X[indices[split:]],
        X[indices[:split]],
        y[indices[split:]],
        y[indices[:split]],
    )


def get_confusion_matrix(y_true, y_pred):
    TP = np.sum((y_true == 1) & (y_pred == 1))
    TN = np.sum((y_true == 0) & (y_pred == 0))
    FP = np.sum((y_true == 0) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == 0))
    return np.array([[TN, FP], [FN, TP]])


def evaluate_metrics(y_true, y_pred):
    cm = get_confusion_matrix(y_true, y_pred)
    TN, FP, FN, TP = cm.ravel()

    accuracy = (TP + TN) / (TP + TN + FP + FN)
    precision = TP / (TP + FP) if (TP + FP) > 0 else 0
    recall = TP / (TP + FN) if (TP + FN) > 0 else 0
    f1 = (
        2 * (precision * recall) / (precision + recall)
        if (precision + recall) > 0
        else 0
    )

    return accuracy, precision, recall, f1, cm


# 3. PLOTTING FUNCTIONS
def plot_decision_boundary(X, y, model, title):
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, Z, alpha=0.3, cmap="RdBu")

    plt.scatter(
        X[y == 0, 0],
        X[y == 0, 1],
        color="red",
        marker="o",
        edgecolors="k",
        label="Class 0 (Test)",
    )
    plt.scatter(
        X[y == 1, 0], X[y == 1, 1], color="blue", marker="x", label="Class 1 (Test)"
    )

    plt.title(title)
    plt.xlabel("Feature 1 (Standardized)")
    plt.ylabel("Feature 2 (Standardized)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()


def plot_cost_history(cost_history):
    plt.figure(figsize=(8, 4))
    plt.plot(cost_history, color="purple", linewidth=2)
    plt.title("MLP Training: Cost vs. Epochs")
    plt.xlabel("Epochs")
    plt.ylabel("Log Loss (Cost)")
    plt.grid(True, alpha=0.3)
    plt.show()


def plot_confusion_matrix(cm, class_names, title="Confusion Matrix"):
    fig, ax = plt.subplots(figsize=(5, 4))
    im = ax.imshow(cm, interpolation="nearest", cmap="Blues")
    ax.figure.colorbar(im, ax=ax)

    ax.set(
        xticks=np.arange(cm.shape[1]),
        yticks=np.arange(cm.shape[0]),
        xticklabels=class_names,
        yticklabels=class_names,
        title=title,
        ylabel="Actual Label",
        xlabel="Predicted Label",
```

```python
    )

    thresh = cm.max() / 2.0
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(
                j,
                i,
                format(cm[i, j], "d"),
                ha="center",
                va="center",
                color="white" if cm[i, j] > thresh else "black",
                fontsize=14,
            )

    fig.tight_layout()
    plt.show()


# 4. MAIN EXECUTION
def main():
    print("-" * 50)
    print(" MULTI-LAYER PERCEPTRON (NON-LINEAR CLASSIFICATION)")
    print("-" * 50)

    # 1. Generate Non-Linear Data (Two Moons)
    print("Generating 'Two Moons' dataset...")
    X, y = make_moons(n_samples=1000, noise=0.15, random_state=42)

    # 2. Split and Standardize
    X_train, X_test, y_train, y_test = train_test_split_custom(X, y, test_size=0.2)

    # Standardize data (Neural networks prefer scaled data)
    mean_val = np.mean(X_train, axis=0)
    std_val = np.std(X_train, axis=0)
    X_train = (X_train - mean_val) / std_val
    X_test = (X_test - mean_val) / std_val

    # 3. Train MLP
    # Architecture: 2 inputs -> 4 hidden neurons -> 1 output
    print(
        f"\nTraining MLP (2 Inputs -> 4 Hidden -> 1 Output) on {len(X_train)} samples..."
    )
    mlp = MultiLayerPerceptron(
        input_size=2, hidden_size=4, output_size=1, learning_rate=0.2, epochs=5000
    )
    mlp.fit(X_train, y_train)

    # 4. Predict & Evaluate
    y_pred = mlp.predict(X_test).flatten()
    acc, prec, rec, f1, cm = evaluate_metrics(y_test, y_pred)

    print("\n" + "=" * 40)
    print("EVALUATION REPORT")
    print("=" * 40)
    print(f"Accuracy:  {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall:    {rec:.4f}")
    print(f"F1 Score:  {f1:.4f}")
    print("\nConfusion Matrix:")
    print(cm)

    # 5. Visualizations
    plot_cost_history(mlp.cost_history)
    plot_confusion_matrix(
        cm, class_names=["Moon 0", "Moon 1"], title="MLP Confusion Matrix"
    )

    # Plot decision boundary specifically over the test set
    plot_decision_boundary(
        X_test, y_test, mlp, title="MLP Decision Boundary (Test Data)"
    )


if __name__ == "__main__":
    main()
```

3 Results and Discussion

3.1 Single Layer Perceptron (Linear Classification)

The SLP was tested on the California Housing binary classification task. Since the dataset is not perfectly linearly separable, the errors per epoch fluctuated rather than dropping to absolute zero.

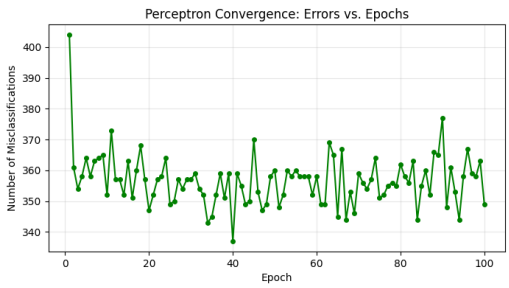
```
 SINGLE LAYER PERCEPTRON (SLP) CLASSIFICATION

Loading California Housing Dataset...
Dataset Shape: (2000, 8)
Target distribution: [ 999 1001]
```

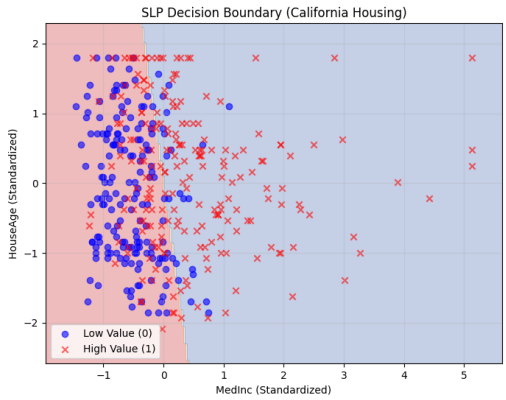
```

Training Perceptron...
========================================
EVALUATION REPORT
========================================
Accuracy:  0.7825
Precision: 0.8508
Recall:    0.7196
F1 Score:  0.7797
Confusion Matrix:
[[159  27]
 [ 60 154]]
```

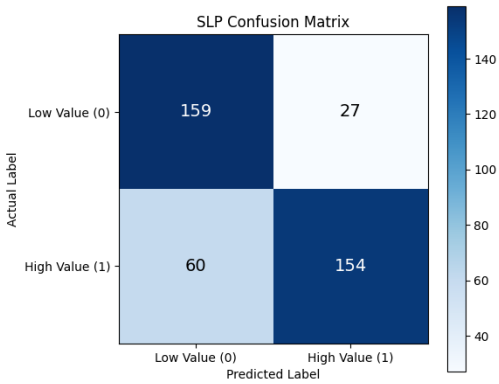
The SLP achieved an accuracy of 78.25%. Figure 1 illustrates the convergence behavior and the learned decision boundary. The straight boundary confirms that the SLP can only isolate linearly separable regions, contributing to the instances of false positives and false negatives seen in the confusion matrix.



(a) Errors per Epoch



(b) Linear Decision Boundary



(c) Confusion Matrix

Figure 1: Single Layer Perceptron Performance Overview

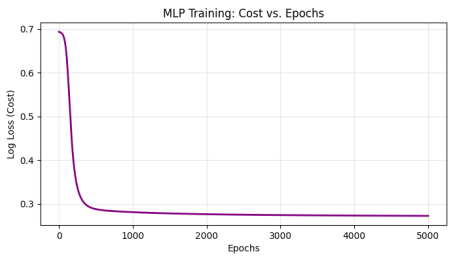
3.2 Multi-Layer Perceptron (Non-Linear Classification)

The MLP (architecture: 2 inputs → 4 hidden neurons → 1 output) was trained on the non-linear "Two Moons" dataset.

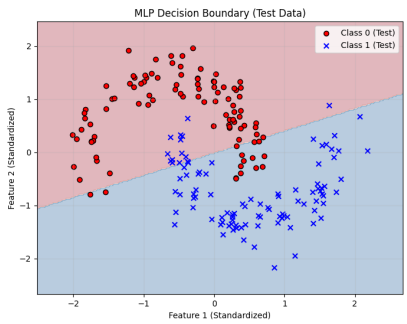
```
 MULTI-LAYER PERCEPTRON (NON-LINEAR CLASSIFICATION)
Generating 'Two Moons' dataset...

Training MLP (2 Inputs -> 4 Hidden -> 1 Output) on 800 samples...
Epoch 0 | Cost: 0.6933
Epoch 1000 | Cost: 0.2808
Epoch 2000 | Cost: 0.2761
Epoch 3000 | Cost: 0.2741
Epoch 4000 | Cost: 0.2730
========================================
EVALUATION REPORT
========================================
Accuracy:  0.8600
Precision: 0.8600
Recall:    0.8600
F1 Score:  0.8600
Confusion Matrix:
[[86 14]
 [14 86]]
```

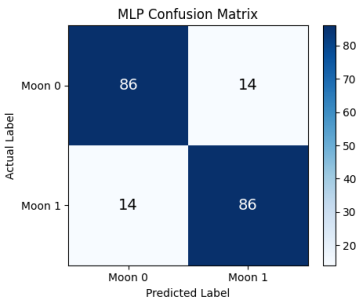
The MLP achieved a balanced accuracy, precision, and recall of 86.00%. The cost history demonstrates a smooth convergence, indicating successful backpropagation. Figure 2 visually highlights the MLP's ability to bend the decision boundary through the hidden layer transformations, successfully separating the crescent shapes that an SLP would fail to capture.



(a) Cost (Log Loss) vs. Epochs



(b) Non-Linear Decision Boundary



(c) Confusion Matrix

Figure 2: Multi-Layer Perceptron Performance Overview

4 Conclusion

This experiment successfully demonstrated the distinction between linear and non-linear classification models.

- The **Single Layer Perceptron** is highly efficient but structurally limited to linearly separable datasets. Its failure to converge perfectly on real-world housing data illustrates the limitation of single-layer architectures.
- The **Multi-Layer Perceptron** overcomes these limitations. By introducing hidden neurons and non-linear Sigmoid activations, it successfully maps complex geometric boundaries, as proven by its strong performance on the Two Moons dataset.

References

[1] C. M. Bishop, *Pattern recognition and machine learning.* springer, 2006.

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning.* MIT press, 2016.